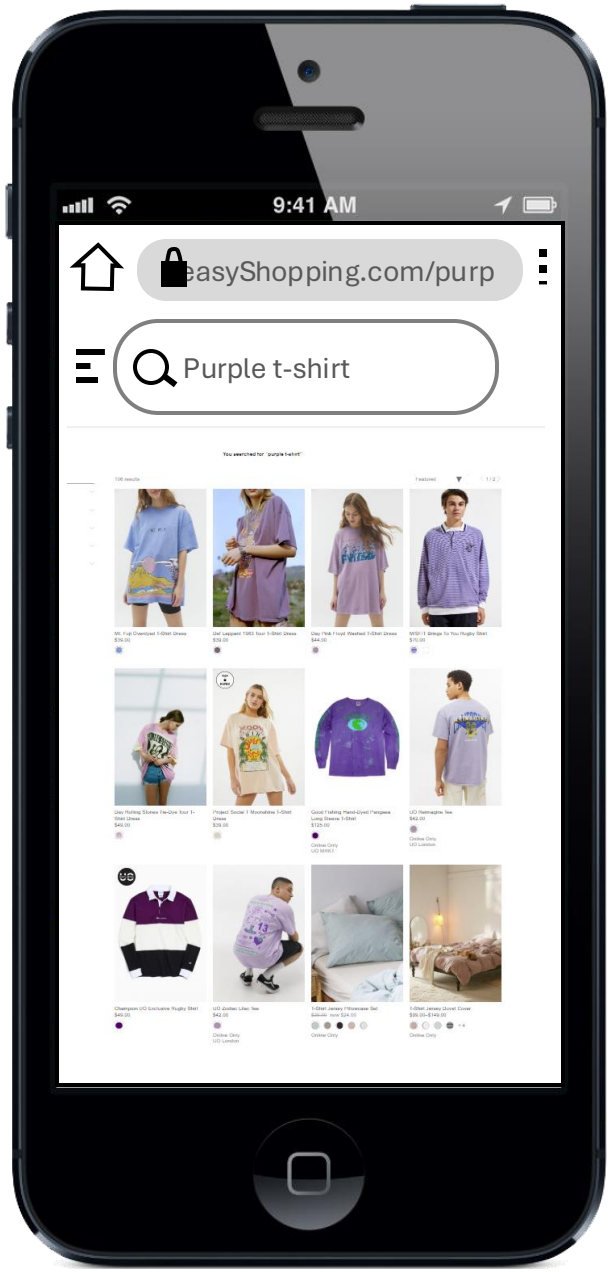


PROJECTE 2
Etiquetador

SS: Sessió de Suport

Project 2 - Part 1

Artificial Intelligence

2025-2026

Universitat Autònoma de Barcelona

1. Recordem...

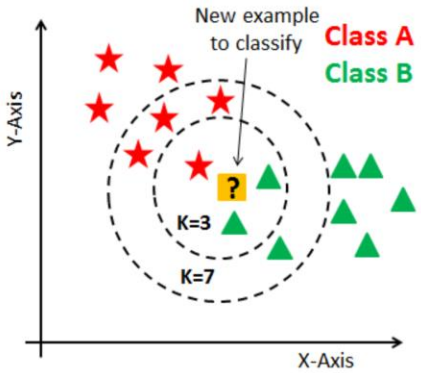
En aquesta part continuem treballant per resoldre el problema d'**etiquetatge d'una imatge**.

Ens centrarem en l'etiquetatge de la forma de la roba implementant l'algorisme **KNN**.

Mètode de classificació supervisada

K-means

KNN



2. Fitxers requerits

Recordeu que per aquesta segona part de la pràctica necessitareu:

1. 📁 **Images:** Carpeta que conté el set d'imatges que utilitzarem.

   A. gt.json: Fitxer que conté informació sobre la classe d'imatges a entrenar (Train).

   B. 📁 Train: Carpeta amb el set d'imatges que utilitzarem com a set d'entrenament. Sobre elles **(train set)** trobarem informació sobre quina classe pertany en el fitxer **gt.json**.

   C. 📁 Test: Carpeta amb el set d'imatges que volem etiquetar **(test set)**.

2. 📁 **Test:** Carpeta que conté els fitxers necessaris per realitzar les proves en els scripts de prova.
   test_cases_knn.pkl

3. **utils.py:** Conté functions ja implementades que podeu utilitzar.

4. **utils_data.py:** Conté un seguit de functions necessaries per obrir i tractar les imatges.

5. **KNN.py:** Fitxer on programareu les funcions necessàries per implementar K-NN per l'etiquetatge de roba.

6. **TestCases_knn.py:** Arxiu amb el qual comproveu si les funcions de KNN.py donen el resultat esperat.

2. Implementació KNN

Per implementar el KNN haureu de programar les següents quatre funcions:

1. _init_train
2. get_k_neighbours
3. get_class
4. Predict

Aclariment: Per a realitzar la classificació supervisada de la forma de les imatges farem servir com a característiques els **píxels de la imatge** després de transformar-la a **escala de grisos**.

2.1 _init_train

Funció que inicialitza les dades d'entrenament.

```
def _init_train (self, train_data):
    implementació
```

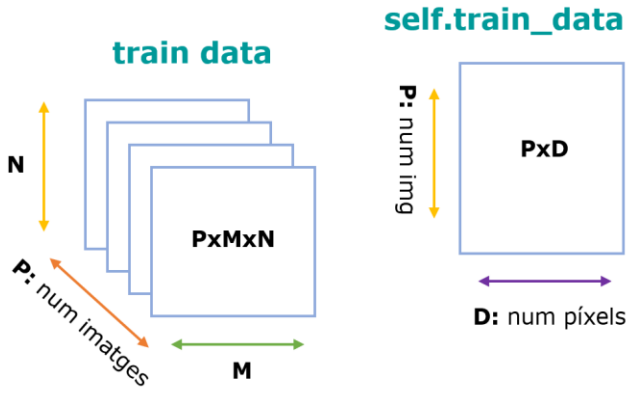
Paràmetre: matriu **PxMxN** corresponent a P imatges en **escala de grisos**

Implementació:

- S'assegura que train_data tingui format **float**.
- N'extreu les característiques.
- Assigna el conjunt d'entrenament a la matriu **self.train_data** amb forma **PxD**. (P punts en l'espai dimensional D)

Nombre d'imatges

Nombre de píxels de cada imatge



2.2 get_k_neighbours

Funció que donada una matriu test_data, calcula els k veïns més propers a cada punt (fila) de test_data a self.neighbors.

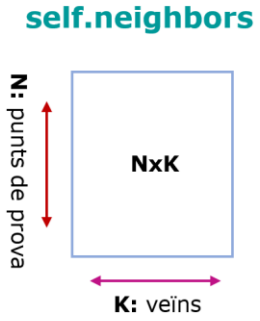
```
def get_k_neighbours (self, test_data, k):
    implementació
```

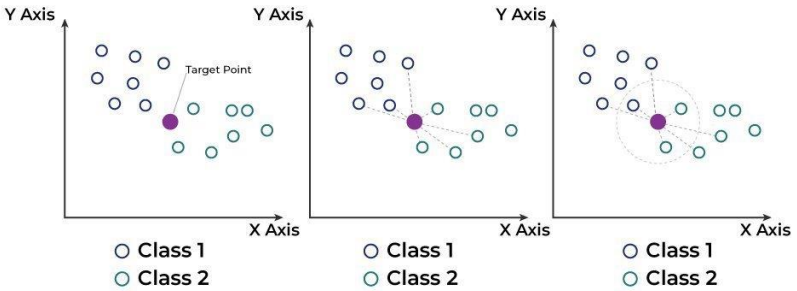
Paràmetres:

- **test_data:** matriu que ha de ser ajustada a una matriu NxD (N punts en un espai dimensional D)
- **k:** nombre de veïns a analitzar

Implementació:

- Canvia les **dimensions** de les imatges (de la mateixa manera que amb train_data).
- Calcula **distància** entre les mostres del test_data i train_data.
- Guarda a la variable self.neighbors les **K etiquetes** de les **imatges més pròximes** per a cada mostra del test.
- Es crea la matriu **self.neighbors** (NxK).





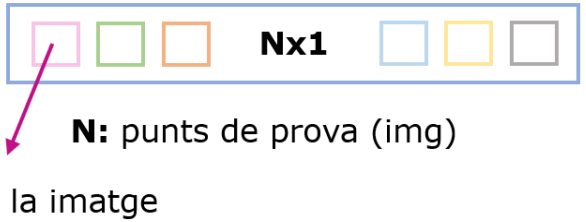
*Pista: Recomanem utilitzar la funció **cdist**, de la llibreria scipy.spatial.distance. Per tal d'evitar un cost computacional elevat.

2.3 get_class

Funció que comprova quina és l'etiqueta que més vegades ha aparegut a la variable de classe neighbors per a cada imatge del conjunt de test.

```
def get_class (self):
    implementació
    return array
```

Retorna: **Matriu numpy** de Nx1 elements.



Implementació:

- Per a cada fila de self.neigbors, obté el valor més votat (la classe a la qual pertany aquella fila).
- Retorna un array amb aquesta classe per a cada imatge.

*Aclariment: Aquest array tindrà tants elements com punts s'hauran entrat a la funció predict.

2.4 predict

Funció en la que fem la predicció de la classe a la qual pertany.

```
def predict (self, test_data, k):
    implementació
    return vector
```

Paràmetres:

- **test_data:** array que ha de ser ajustat a una matriu NxD (N punts en un espai dimensional D)
- **k:** nombre de veïns a considerar

Retorna: La sortida de **get_class**, un vector **Nx1** amb la **forma predita** per a cada imatge de prova.

Implementació:

- Crida la funció get_k_neighbours per buscar els **veïns més propers**.
- Crida la funció get_class per retornar la **classe més representativa**.

3. Proveu el vostre codi

```python
if __name__ == '__main__':


    # import images in grayscale
    train_imgs_grayscale, train_class_labels, _, test_imgs_grayscale, test_class_labels, _ = read_dataset(
        root_folder='./images/', gt_json='./images/gt.json', with_color=False
    )
    knn = KNN(train_imgs_grayscale, train_class_labels)
    knn.get_k_neighbours( test_data: [test_imgs_grayscale[4]],  k: 7)
    print(knn.neighbors)
```

Carregar les imatges de la carpeta "Images"

Donem al KNN el conjunt d'entrenament

Introduim al Knn la 4a imatge del conjunt test amb K = 7

Comprovem quins veins troba

3. Proveu el vostre codi

```
test_imgs_grayscale = read_one_img( img_name: "my_path/imagename.png",  w: 60,  h: 80,  with_color: False)

knn.get_k_neighbours( test_data: [test_imgs_grayscale],  k: 7)

print(knn.neighbors)
```

Carregar la imatge que volem testejar, amb una mida de 60x80 en escala de grisos (with_color: False)

Introduim al Knn la imatge amb K = 7

Comprovem quins veins troba

5. Avaluació per qüestionaris

Per a l'avaluació d'aquesta part de la pràctica haureu de contestar un seguit de qüestionaris del Campus Virtual.

- **Proj2.1-Q1**: qüestionari corresponent a la **SO** amb preguntes tipus test i avaluació de les funcions *_init_train, get_k_neughbours, get_class i predict.*
- **Proj2.1-Q2**: qüestionari corresponent a la **SS** amb preguntes tipus test i avaluació on haureu d'executar l'algoritme KNN amb imatges externes.
- **Proj2-Par1**: des de Divendres, 20 de Març 17:00h fins Diumenge, 22 de Març 23:55h. Avaluació de totes les funcions anteriors.

ATENCIÓ! és important que tingueu en compte els següents punts:

1. La **correcció** del codi es fa de manera **automàtica**, per tant, assegureu-vos de penjar els mètodes amb la nomenclatura i format correctes. Si no ho poseu bé la nota de l'exercici serà un 0.

2. El codi està sotmès a **detecció automàtica** de plagis durant la correcció.

3. Per evitar que el codi entri en bucles infinits hi ha un **límit de temps** per a cada exercici, per tant si les vostres funcions triguen massa les considerarà incorrectes.

Recordeu el que es diu a la **guia docent** sobre **copiar o deixar copiar ....**

Sense perjudici d'altres mesures disciplinàries que s'estimin oportunes, i d'acord amb la normativa acadèmica vigent, les **irregularitats comeses per l'alumnat** que puguin conduir a una variació de la qualificació es qualificaran amb un zero (0). Les activitats d'avaluació qualificades d'aquesta forma i per aquest procediment no seran recuperables. Si és necessari superar qualsevol d'aquestes activitats d'avaluació per aprovar l'assignatura, aquesta assignatura quedarà suspesa directament, sense oportunitat de recuperar-la en el mateix curs. Aquestes irregularitats inclouen, entre d'altres:

- còpia total o parcial d'una pràctica, informe, o qualsevol altra activitat d'avaluació;
- deixar copiar;
- ús no autoritzat i/o no referenciat de la IA (p. ex,Copilot, ChatGPToequivalents) per a resoldre exercicis, pràctiques i/o qualsevol altra activitat avaluable;
- presentar un treball de grup no fet íntegrament pels membres del grup;
- presentar com a propis materials elaborats per un tercer, encara que siguin traduccions o adaptacions, i en general treballs amb elements no originals i exclusius de l'alumnat.
- tenir dispositius de comunicació (com telèfons mòbils, smart watches, etc.) accessibles durant les proves d'avaluació teòric-pràctiques individuals (exàmens).

En resum: **copiar, deixar copiar o plagiar** en qualsevol de les activitats d'avaluació equival a un SUSPENS amb **nota inferior o igual a 3,0.**